

PharmaMobile - Evidencias Sesión 5

Reto 01: reorganización del módulo de Productos en capas Clean + MVVM con Koin

Rama: **feature/clean-mvvm** - Repositorio: github.com/Plasencia2007/pharmamobile

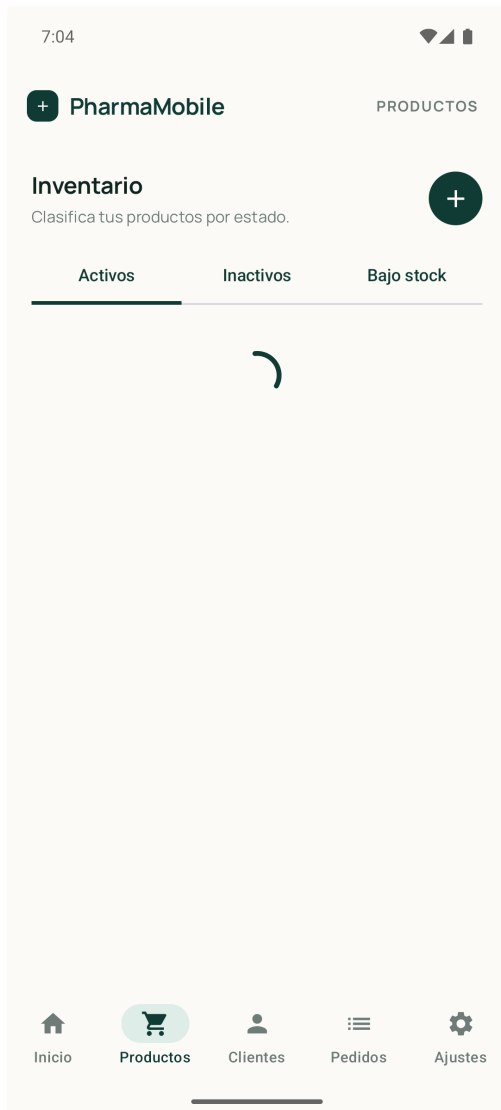
1. Arquitectura implementada

- domain/repository/ProductoRepository - interfaz con registrar() y listar() suspend, en lenguaje de negocio.
- data/repository/ProductoRepositoryEnMemoria - implementación en memoria, delay simulado, asigna el id.
- domain/usecase/RegistrarProductoUseCase - concentra la validación (antes en ProductoValidator) y devuelve Result<Producto>.
- Producto - reglas de negocio propias: estaInactivo() y requiereReposicion() / STOCK_MINIMO.
- presentation/producto/ProductoUiState - 4 fases del inventario (Cargando, SinProductos, ConProductos, Error) más estado del formulario.
- presentation/producto/ProductoViewModel - StateFlow de solo lectura vía asStateFlow(), viewModelScope.
- ProductoScreen - sin remember de negocio ni llamadas al validador; resuelve la lista con un when exhaustivo.
- di/AppModule + di/PlatformModule (expect/actual) - Koin ensambla repositorio (single), caso de uso y ViewModel.
- MainApplication + android:name en el manifiesto; initKoin() invocado desde iOSApp.swift.
- App.kt envuelto en KoinContext; Screen.Productos inyecta el ViewModel con koinViewModel().

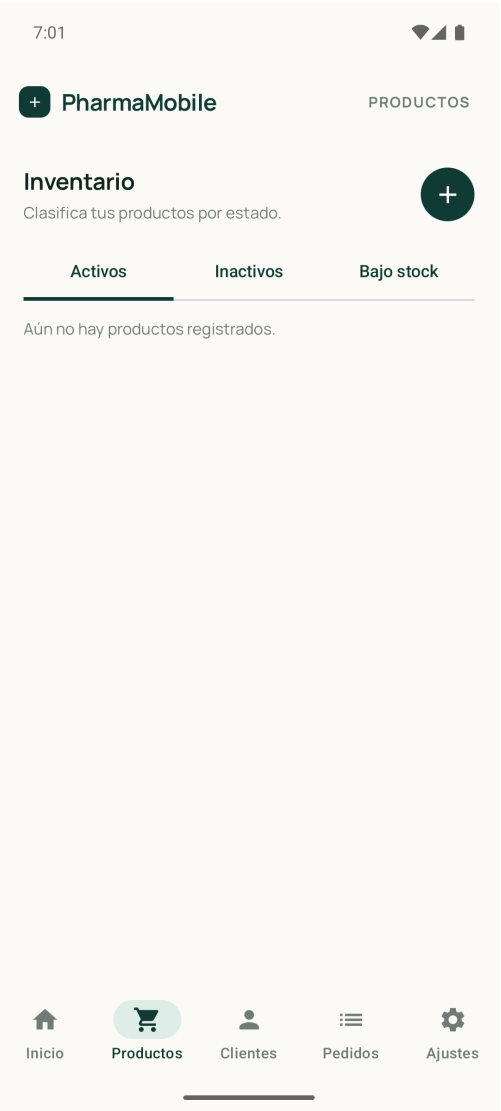
Nota: ProductoService/ResultadoProductos (ejercicio de corrutinas y Flow de la Sesión 3) se conservaron sin cambios de comportamiento, solo se actualizó la referencia a la clase renombrada, para no romper ProductoFlowAndroidHostTest.

2. Los 4 estados de la pantalla de Productos - Android

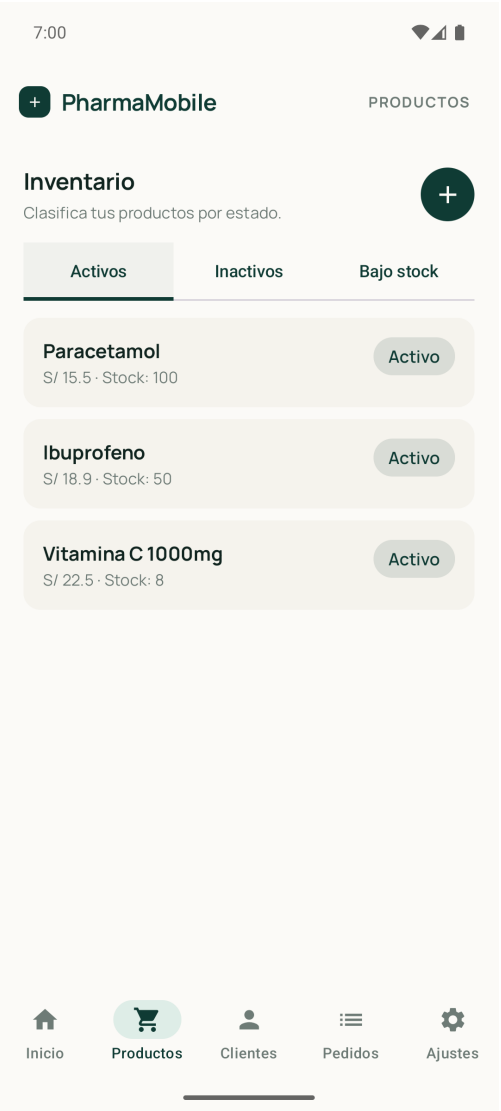
Capturado en emulador Pixel 8 (API 36). El estado iOS no pudo evidenciarse en esta sesion de trabajo porque el entorno de ejecucion es Windows y compilar/ejecutar la app de iOS requiere Xcode en macOS; el modulo shared si se compilo exitosamente para iosSimulatorArm64 (ver seccion 5), confirmando que el codigo Kotlin/Native y la integracion de Koin no tienen errores de compilacion.



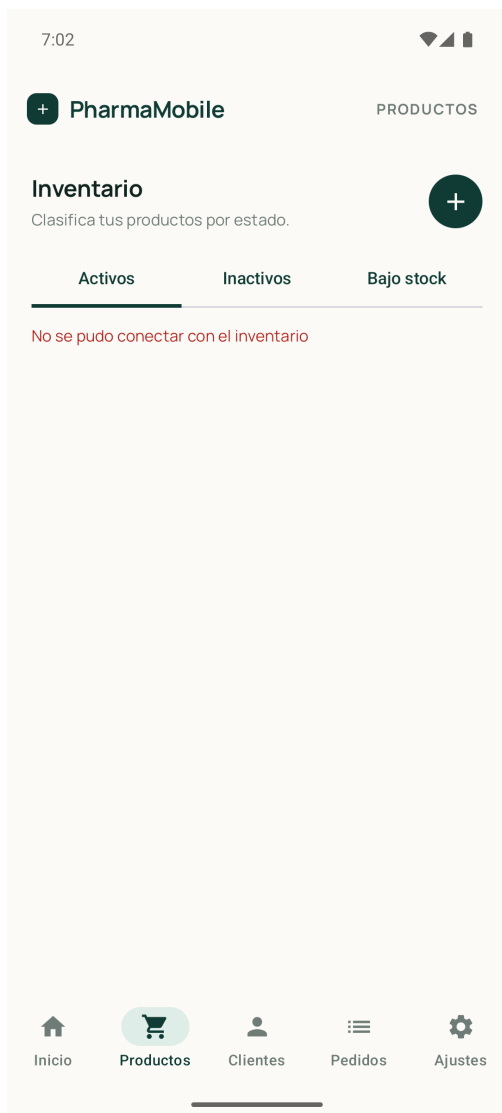
Estado Cargando - CircularProgressIndicator mientras el ViewModel espera `productoRepository.listar()`.



Estado SinProductos - repositorio forzado temporalmente a devolver una lista vacia.



Estado ConProductos - pestana Activos, incluye el producto registrado en vivo (Vitamina C 1000mg).



Estado Error - repositorio forzado temporalmente a lanzar una excepcion, capturada por runCatching en el ViewModel.

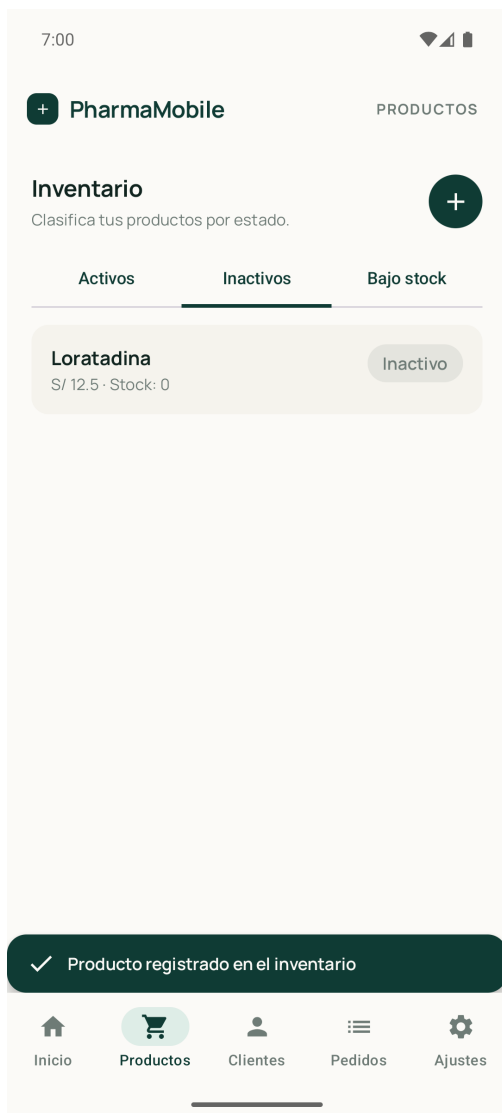
3. Flujo de registro (caso de uso + repositorio + Koin end-to-end)



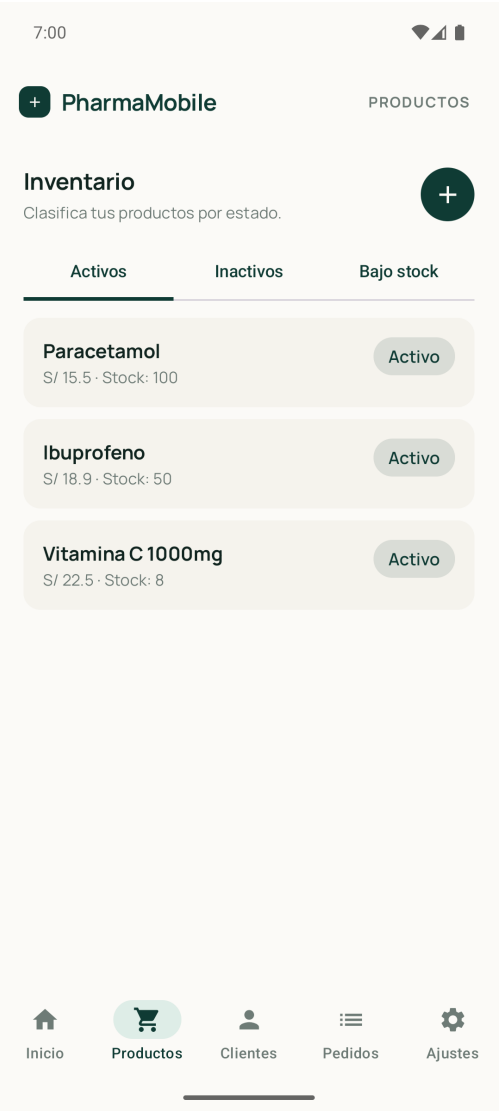
El boton + abre el ModalBottomSheet con el formulario de registro.



Formulario completado: nombre, precio y stock.



Snackbar de exito tras registrar; RegistrarProductoUseCase valido y ProductoRepositorioEnMemoria asigno el id.



El producto registrado persiste al navegar a Clientes y volver a Productos (repositorio Koin declarado como single).

4. Arbol de paquetes de shared

```
pe/edu/upeu/pharmamobile/  
  App.kt  
  Greeting.kt - GreetingUtil.kt - Platform.kt (expect)  
  data/repository/  
    ProductoRepositorioEnMemoria.kt  
  di/  
    AppModule.kt  
    PlatformModule.kt (expect)  
  domain/  
    demo/DemoResultadoProducto.kt  
    model/ Cliente - DetallePedido - EstadoPedido - Pedido - Producto  
    repository/ProductoRepository.kt (interfaz)  
    result/ResultadoProductos.kt  
    service/ProductoService.kt  
    usecase/RegistrarProductoUseCase.kt  
  navigation/Screen.kt  
  presentation/  
    cliente/ ClienteScreen - ClienteValidator  
    components/ DashedBorder - DisplayHeading - ErrorSummary - FormButton - PharmaChip -  
PharmaField - PharmaTopBar  
    config/ConfigScreen.kt  
    inicio/InicioScreen.kt  
    pedido/ PedidoScreen - PedidoValidator  
    producto/ ProductoScreen - ProductoUiState - ProductoViewModel  
    theme/ Color - Theme - Type  
  
androidMain/pe/edu/upeu/pharmamobile/  
  Platform.android.kt  
  di/PlatformModule.android.kt (actual)  
  
iosMain/pe/edu/upeu/pharmamobile/  
  MainViewController.kt - Platform.ios.kt  
  di/ PlatformModule.ios.kt (actual) - KoinIos.kt
```

5. Ejecucion de pruebas

Comando: `./gradlew :androidApp:assembleDebug :shared:testAndroidHostTest`

```
> Task :shared:compileAndroidMain
> Task :shared:compileAndroidHostTest
> Task :shared:testAndroidHostTest
```

```
pe.edu.upeu.pharmamobile.ProductoFlowAndroidHostTest > evidenciaCorrutinasYFlow PASSED (4.135s)
pe.edu.upeu.pharmamobile.SharedCommonTest > example PASSED (0.000s)
pe.edu.upeu.pharmamobile.SharedLogicAndroidHostTest > clienteTelefono PASSED (0.001s)
```

```
BUILD SUCCESSFUL in 9s
32 actionable tasks: 2 executed, 30 up-to-date
```

Tambien se compilo el target iOS del modulo shared (sin errores) para confirmar que el codigo Kotlin/Native y la integracion de Koin son validos, aunque no se pudo ejecutar en un simulador real por no disponer de macOS/Xcode en este entorno:

Comando: `./gradlew :shared:compileKotlinIosSimulatorArm64 -> BUILD SUCCESSFUL`

6. Lista de cotejo (autoevaluacion)

N	Criterio	Cumple
1	shared presenta domain/repository, domain/usecase, data/repository y di	Si
2	Ninguna clase de domain importa Compose, Ktor, Android o iOS	Si
3	Producto incorpora al menos una regla de negocio propia	Si - estaInactivo() / requiereReposicion()
4	ProductoRepository es interfaz en domain; implementacion en memoria en data, suspend	Si
5	El id lo asigna el repositorio, no la pantalla	Si
6	RegistrarProductoUseCase concentra las reglas y devuelve Result	Si
7	ProductoUiState modela las 4 fases y el estado del formulario	Si
8	ViewModel expone StateFlow de solo lectura via asStateFlow()	Si
9	ProductoScreen sin remember de negocio ni llamadas al validador	Si
10	Koin ensambla repositorio (single), caso de uso y ViewModel	Si
11	Manifiesto declara android:name=".MainApplication"; iOSApp.swift invoca init() y start()	Si
12	La app arranca y opera en Android e iOS, con evidencia de los 4 estados en data	Parcial - Android completo; iOS solo compilacion (sin
13	Pruebas de commonTest sin fallos; cambios versionados en la rama indicada	Si - rama feature/clean-mvvm, commit b6e9150